

TECHNICAL TEARDOWN

Lobstr Sales Navigator

URL Splitter

Observable behavior, limitations, and a build specification for an adaptive live-count Chrome extension

Prepared	Scope	Evidence
22 July 2026	Public free splitter + Lobstr Split Search documentation	Live black-box tests + supplied 68-URL CSV

Bottom line The free web tool is a deterministic URL combinator, not an adaptive validator. It can preserve a follower filter and append static facet IDs, but it does not prove that every generated child is at or below 2,500 results.

Research note

This is a black-box analysis, not Lobstr source-code reverse engineering. Confirmed observations, vendor-documented behavior, and design inferences are labeled separately.

Executive summary

Lobstr’s public tool accepts a Sales Navigator people-search URL, lets the user select Industry, Function, and/or Seniority, and downloads generated child URLs. In current public testing, it created 68 industry URLs, 26 function URLs, and 10 seniority URLs. Selecting all three created 17,680 URLs, exactly $68 \times 26 \times 10$. This is strong evidence of a static Cartesian-product generator rather than an adaptive count-driven splitter.

1. The generator preserves the original search expression and adds one value per selected facet dimension.
2. The output panel and supplied CSV contain URLs, not live result counts or validation status.
3. The tool’s own published example contains industry buckets of 2,501, 4,001, and 5,501 results, so its public claim of perfectly segmented children should not be interpreted as a $\leq 2,500$ guarantee.
[1]
4. The supplied follower-search export contains 68 unique URLs, each preserving `FOLLOWS_YOUR_COMPANY` and adding one legacy `INDUSTRY` value.
5. A dependable extension must check every child live, accept only verified leaves, and recursively subdivide only the children that remain over the threshold.

Recommended threshold Use 2,500 as the hard ceiling but target about 2,200. The lower planning target provides room for count drift between validation and extraction.

Contents at a glance

Section	Purpose
1	What the public Lobstr tool actually does
2	Black-box findings and the supplied CSV audit
3	How Sales Navigator URLs are transformed
4	Why static splitting produces 600-result and 3.5K-result buckets
5	Difference between the free tool and Lobstr’s paid Split Search
6	Adaptive Chrome extension architecture and algorithm
7	Coverage validation, CSV schema, testing, and acceptance criteria

1. What the public Lobstr tool actually does

1.1 Confirmed user-visible workflow

6. Accept a Sales Navigator people-search URL in a single input field.
7. Let the user choose Industry, Function, Seniority, or multiple dimensions.
8. Generate child Sales Navigator URLs by inserting facet filters into the query expression.
9. Display the total number of generated URLs and enable a Download file button.
10. Download a URL-only file intended for Lobstr's separate Sales Navigator Leads Scraper. [1]

1.2 Observable processing model



No adaptive live-count loop in the free generator output

Figure 1. Observable model of the free public generator.

The public generator produced its category URLs without displaying a live count beside each output. Its marketing page includes a separate worked example table with counts, but that table is not evidence that arbitrary authenticated searches—including `FOLLOWS_YOUR_COMPANY`—are validated live.

Inference boundary The most likely implementation is a client-side or server-assisted query rewriter backed by static facet dictionaries. The internal source code was not available, so the implementation mechanism itself is an inference; the output cardinalities and URL transformations are directly observed.

2. Black-box findings

2.1 Current public test

Selected dimensions	Generated URLs	Observed pattern
Industry only	68	One INDUSTRY value per URL
Function only	26	One FUNCTION value per URL
Seniority only	10	One SENIORITY_LEVEL value per URL
All three	17,680	$68 \times 26 \times 10$ Cartesian combinations

The page copy currently says its example has 45 industry, 18 function, and 10 seniority segments, while the live generator returned 68, 26, and 10. The mismatch indicates the descriptive example and generator dictionaries are not synchronized.

2.2 Why the 17,680 result matters

A smart adaptive splitter would not pre-generate every Industry $\times$ Function $\times$ Seniority combination. It would first split by one dimension, live-check those children, and only add another dimension beneath the children still above the ceiling. Lobstr's 17,680-output behavior demonstrates up-front multiplication, which creates many empty or tiny queries while still offering no proof that a highly concentrated combination is below 2,500.

```
industry_count × function_count × seniority_count  
68 × 26 × 10 = 17,680 generated URLs
```

2.3 Public example contradicts a strict ceiling guarantee

Lobstr's published crab-keyword example lists Hospitality at 2,501 results, Food and Beverage Services at 4,001, and Restaurants at 5,501. Those values exceed the visible-results ceiling. Therefore the free generator performs categorization, not a guaranteed terminal partition. [1]

Practical conclusion Treat every free-tool URL as an unchecked candidate. A child URL becomes export-ready only after your extension records a current count that is safely at or below the chosen threshold.

3. Audit of the supplied follower-search CSV

The attached export was parsed locally. Session-bound values are intentionally not reproduced in this report.

Check	Observed	Meaning
URL rows	68	All rows under the url column
Unique URLs	68	No duplicate URL strings detected
Industry IDs	68	One distinct INDUSTRY ID in each child
Highest supplied legacy ID	100	Static list ends at 100 in this export
Result-count column	Absent	No per-URL live validation evidence

3.1 Decoded child-query shape

```
(
  recentSearchParam:(doLogHistory:true),
  filters:List(
    (type:FOLLOWS_YOUR_COMPANY,
      values:List((id:CF,selectionType:INCLUDED))),
    (type:INDUSTRY,
      values:List((id:1,selectionType:INCLUDED)))
  )
)
```

Every audited child keeps the follower condition and appends one industry value. That explains why the generated URLs are relevant to the follower search. It does not establish that all follower records are covered or that each bucket is under the cap.

3.2 Static industry list observed

```
1, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 23, 24, 25, 27, 31, 32, 34, 35,
36, 37, 38, 39, 40, 41, 42, 43, 44, 47, 49, 51, 54, 56, 57, 60, 62, 65, 66, 67, 68, 71, 74, 75, 76,
78, 79, 80, 81, 84, 86, 88, 90, 91, 92, 93, 94, 95, 96, 99, 100
```

This 68-value list resembles the older/coarser industry set. LinkedIn's Industry V2 taxonomy contains hundreds of nodes and IDs well above 100, while Lobstr's own paid Split Search documentation currently describes 146 industry values. [2][5] The free generator's list should therefore be treated as a maintained implementation detail—not an authoritative, exhaustive taxonomy.

3.3 Parameters and privacy

11. The audited output contains the query, a sessionId value, and viewAllFilters=true.
12. The sessionId is not the LinkedIn authentication cookie, but it is transient search context and should not be treated as a stable identifier.

13. A new extension should never copy, store, or transmit authentication cookies or CSRF tokens.
14. Before exporting, validate whether a canonical URL works without transient page/session parameters; preserve them only when required for that account-bound filter.

4. How the URL transformation works

4.1 Two encoding layers

Sales Navigator people-search URLs contain an outer standard query string. Inside the outer query parameter is a nested Rest.li-style expression using parentheses, List(...), colons, and commas. The nested expression is percent-encoded in the final browser URL.

```
Outer URL
https://www.linkedin.com/sales/search/people?query=<encoded-expression>

Decoded inner expression
(filters:List((type:INDUSTRY,values:List((id:1,selectionType:INCLUDED))),keywords:crab)
```

4.2 Safe rewriting sequence

15. Parse the outer URL with URL and URLSearchParams.
16. Read and decode query exactly once.
17. Parse the nested expression into a token tree or small abstract syntax tree; do not edit it with a broad regular expression.
18. Preserve unknown keys and all existing filters, including FOLLOWS_YOUR_COMPANY.
19. Insert one INCLUDED value for the chosen partition dimension.
20. Serialize the inner expression deterministically and percent-encode it exactly once.
21. Reset page to 1 or remove the page parameter; validate the resulting child before storing it.

4.3 Partition semantics

Construction	Likely search meaning	Splitter implication
Values within one dimension	Usually OR	Industry A or Industry B in one URL is not two partitions
Different dimensions	Usually AND	Industry A and Seniority Director narrows the same set
One child per facet value	Partition candidate	Required for independent count validation
All combinations up front	Cartesian product	Produces 17,680 queries in the Lobstr test

Parser requirement The rewriter must be idempotent: applying the same facet twice must not duplicate it, and parsing then serializing an untouched URL must preserve its meaning.

5. Why the buckets are uneven

Static categories are not equal-size bins. A broad follower population can have thousands of people in a common industry and only hundreds in a narrow one. This is why one Lobstr URL can show roughly 600 results while another shows 3.5K or more.

Cause	Effect
Skewed population	Common industries/functions contain a disproportionate share of the root audience.
No feedback loop	The free output contains no count-driven decision to subdivide an oversized child.
Static dictionaries	A fixed category list can be incomplete or drift from LinkedIn's current filter taxonomy.
Missing/unknown attributes	People or current employers without a usable facet value may not fall into an enumerated child.
Rounded counts	Labels such as 3.5K+ and 100K+ are lower bounds, not exact totals.
Live data changes	Counts can move between validation and later lead extraction.

5.1 Coverage cannot be inferred from URL count

Generating 68 industry URLs does not prove the union equals the original 100K follower search. A reliable system needs both leaf-size validation and coverage accounting. When LinkedIn reports only rounded totals, the extension should label coverage as estimated or unproven rather than claim 100%.

Conservative rule Accept a leaf only when the observed count's upper bound is $\leq 2,500$. A value such as 2.5K+ is over-cap/unknown, not safe.

6. Free splitter versus Lobstr’s paid Split Search

Lobstr documents a different feature inside its paid Sales Navigator Leads Scraper. That feature is called Split Search and should not be confused with the free URL generator. [2]

Property	Free URL generator	Paid Split Search
Where it runs	Public web page	Inside Lobstr Leads Scraper
Output	Child URL file	Leads collected under one run
Dimensions	Industry / Function / Seniority	Seniority 10 → Function 26 → Industry 146
Selection	User selects dimensions	Chooses first unused dimension with enough values
Sizing target	No live threshold shown in output	Estimates required buckets using total ÷ ~2,000
Recursion	No evidence	Explicitly none
Oversized child	Remains unchecked	Paged to 2,500; excess can be silently missed
Scale guidance	Marketing says no caps	Docs say best up to about 50,000

The paid algorithm estimates the number of buckets needed as total results divided by about 2,000, then chooses Seniority, Function, or Industry if the dimension is unused and has enough distinct values. It creates at most 95 sub-searches and does not recursively split a child that still exceeds 2,500. [2]

Design lesson Copy the useful idea—choose a dimension based on required capacity—but add recursion, live child validation, and explicit unresolved states. Those three changes address the failure mode affecting your 100K follower search.

7. Proposed adaptive Chrome extension

7.1 Design goal

Given the active Sales Navigator people search, generate the smallest practical set of child URLs whose most recently observed counts are safely under the ceiling, while preserving all original filters and recording enough evidence to audit the split.

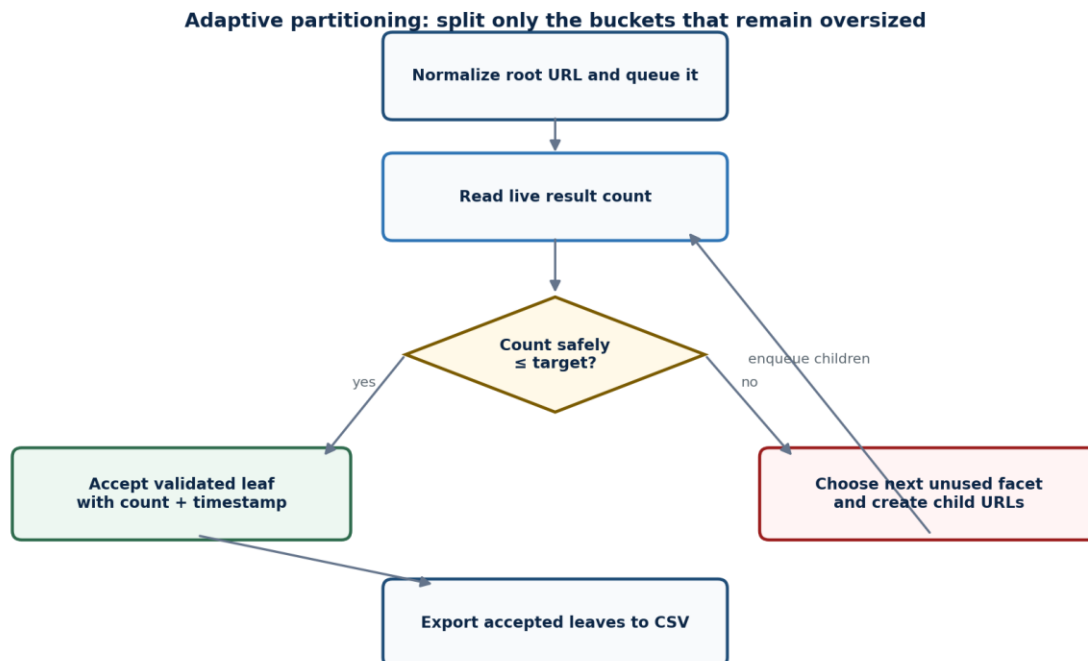


Figure 2. Recursive, count-driven partitioning workflow.

7.2 Recommended extension components

Module	Responsibility
URL codec	Parses/serializes the nested query and preserves unknown filters.
Facet provider	Reads currently available facet values and IDs; supports versioned fallback lists.
Count reader	Returns a structured exact/lower-bound/unknown result for one child URL.
Partition planner	Scores dimensions and recursively splits only over-cap nodes.
Job queue	Single controlled worker with pause, resume, retry, and checkpointing.
Persistence	Stores job tree and validation metadata in chrome.storage.local or IndexedDB.
CSV exporter	Exports accepted leaves plus count and audit columns.
UI	Start, pause, resume, stop, inspect unresolved nodes, and download results.

7.3 Chrome MV3 placement

- 22. Content script: reads the current Sales Navigator page, observes count/facet state, and displays a small in-page panel.
- 23. Service worker: owns the durable queue and communicates with the content script; it cannot directly inspect the page DOM.
- 24. Local storage: persists the partition tree so a browser restart does not discard progress.
- 25. Downloads permission: creates the final CSV locally. Avoid broad <all_urls> permission and avoid remote code.
- 26. Host permission: limit to the precise Sales Navigator origin/path required by the extension.

8. Adaptive splitting algorithm

8.1 Core loop

```

queue = [normalize(currentSearchUrl)]
accepted = []
unresolved = []

while queue is not empty:
    node = queue.pop()
    observation = liveCount(node.url)

    if observation.is_zero:
        mark(node, 'empty')
    elif observation.upper_bound <= TARGET:
        accepted.append(validate(node, observation))
    else:
        dimension = chooseNextDimension(node, observation)
        if dimension is None:
            unresolved.append(mark(node, 'cannot_split_safely'))
        else:
            children = oneChildPerFacetValue(node, dimension)
            persist(node, children, observation)
            queue.extend(children)

exportCsv(accepted, unresolved)

```

8.2 Dimension-selection score

Do not always choose Industry first. For each unused candidate dimension, estimate how well it reduces the largest child and how completely it covers the parent. Prefer the dimension with enough valid values, low observed maximum bucket size, low overlap risk, and good facet coverage.

Factor	Measurement	Preference
Capacity	Number of usable values relative to $\text{parent_count} \div \text{target}$	Higher is better
Balance	Largest child count / parent count	Lower is better
Coverage	Sum/union evidence versus parent	Closer is better
Reuse	Dimension already fixed in parent	Disqualify
Request cost	Children required to evaluate	Lower is better
Stability	Known taxonomy and count reliability	Higher is better

8.3 Suggested dimension order for a follower search

Use live facet information rather than a fixed order. A reasonable initial candidate set is geography, industry, function, seniority, and company headcount. Geography often has useful hierarchical fallback (country → region), while Industry/Function/Seniority are convenient but can be highly skewed. Only use a new dimension beneath an oversized parent.

Avoid Lobstr's explosion For a 3.5K industry bucket, test Seniority children only beneath that one industry. Do not generate all 17,680 global combinations.

9. Live-count reader specification

9.1 Return an interval, not just an integer

Displayed	Minimum	Maximum	Kind	Decision
600	600	600	exact	Potentially acceptable
2,500	2,500	2,500	exact	At hard ceiling; prefer margin
2.5K+	2,500	unknown	lower_bound	Do not accept
3.5K+	3,500	unknown	lower_bound	Split again
100K+	100,000	unknown	lower_bound	Root/over-cap
Unavailable	unknown	unknown	error	Retry or unresolved

9.2 Count-reader implementation choices

Mode	Approach	Trade-off
Visible-page reader	Navigate a controlled Sales Navigator tab and read the rendered count.	Slower but decoupled from private response schemas; still automated activity.
Undocumented network adapter	Read the authenticated response used by the current page.	Faster but fragile, policy-sensitive, and must never export credentials.
Manual validation mode	Extension opens each candidate; user confirms count.	Lower automation but high effort for large trees.

Keep the count-reader behind a narrow interface so it can be replaced when LinkedIn changes its page. Never hard-code authentication cookies into extension source, logs, CSV files, or remote telemetry.

9.3 Retry and stop conditions

27. Pause immediately on sign-in redirects, CAPTCHA/challenge pages, 429 responses, or account-restriction warnings.
28. Use one active count check at a time and checkpoint after every observation.
29. Do not convert an error into a zero count; errors remain unresolved.
30. Store `checked_at` and a count fingerprint so stale observations can be revalidated later.

10. Coverage and correctness

10.1 Required invariants

31. Every exported accepted URL has a successful count observation from the current run.
32. No accepted leaf has an observed upper bound above the target.
33. Every over-cap node is either split further or exported separately as unresolved—not silently dropped.
34. Existing filters, especially FOLLOWS_YOUR_COMPANY, remain in every descendant.
35. The same partition path cannot be enqueued twice.
36. Zero-result children may be discarded only after a valid zero observation.
37. Coverage is labeled proven, estimated, or unproven; it is never implied by the number of URLs.

10.2 Count conservation

When counts are exact and children are known to be mutually exclusive, compare the sum of child counts with the parent. A mismatch may signal missing facet values, overlap, volatile data, or query-rewrite corruption. With rounded counts, use interval comparisons and retain an uncertainty flag.

```
exact case:     $\sum \text{child\_count} == \text{parent\_count}$   
interval case: parent interval intersects  $\sum$  child intervals  
otherwise:    coverage_status = 'unproven'
```

10.3 Dedupe after lead extraction

Even well-designed partitions can overlap because of LinkedIn profile semantics, multiple current roles, or taxonomy behavior. After your separate extractor runs, consolidate files and deduplicate by a stable Sales Navigator member identifier or normalized LinkedIn profile URL. Keep the originating partition_id for auditability.

Honesty requirement The extension can guarantee that its saved URLs passed the live size check at a recorded time. It cannot honestly guarantee that LinkedIn's changing, rounded, account-specific search index yielded every follower unless coverage can be independently proven.

11. CSV output contract

Do not export a single url column. Export enough state to reproduce, validate, and resume the job.

Column	Example/type	Why it exists
partition_id	Stable local UUID	Identity and dedupe
parent_id	Parent UUID or blank	Tree reconstruction
depth	0, 1, 2...	Recursion audit
status	accepted / empty / unresolved	Safe downstream selection
dimension	INDUSTRY, FUNCTION...	Partition method
facet_id	LinkedIn value ID	Reproducibility
facet_label	Human-readable label	Inspection
reported_count	Raw text, e.g. 3.5K+	Evidence preservation
count_min	Parsed lower bound	Decision logic
count_max	Parsed upper bound or blank	Safe acceptance
count_kind	exact / lower_bound / error	Uncertainty
threshold	Configured target	Run context
checked_at	ISO 8601 timestamp	Freshness
coverage_status	proven / estimated / unproven	Claim control
url	Canonical child URL	Input to lead extractor
validation_error	Blank or concise reason	Recovery

```
partition_id,parent_id,depth,status,dimension,facet_id,facet_label,reported_count,count_min,count_max,
count_kind,threshold,checked_at,coverage_status,url,validation_error
```

12. Test plan and acceptance criteria

12.1 URL-codec unit tests

38. Round-trip a keyword-only query without semantic changes.
39. Preserve FOLLOWS_YOUR_COMPANY when adding Industry.
40. Add exactly one value and avoid duplicate filters on rerun.
41. Handle existing Industry, Function, and Seniority filters.
42. Reset/remove page while preserving unrelated outer parameters.
43. Encode special characters exactly once.
44. Reject malformed or non-people-search URLs with a clear error.

12.2 Planner integration tests

Scenario	Expected result
Root = 600	Accept root; generate no children
Root = 3.5K+	Choose facet and enqueue children
One child = 2,700	Recursively split only that child
Child = 0	Mark empty after valid observation
Count unavailable	Retry, then unresolved; never accept
All dimensions exhausted	Unresolved with reason
Browser restart	Resume from persisted queue without duplication
Challenge/restriction page	Pause job and surface warning

12.3 Definition of done

45. Every accepted row contains URL, threshold, count evidence, and timestamp.
46. No accepted row is over the configured ceiling.
47. The job is resumable and idempotent.
48. Unchecked, blocked, stale, and over-cap nodes are visible—not hidden.
49. No authentication secret leaves the browser or appears in logs/exports.
50. The extension never claims complete coverage when the evidence is approximate.
51. The exported CSV opens cleanly and the existing lead extractor accepts the url column.

13. Operational and account-risk notes

A live-count Chrome extension is still automation on LinkedIn. LinkedIn states that third-party software and browser extensions that scrape or automate activity are not permitted, and automated activity can result in temporary or permanent account restrictions. [4]

52. There is no request rate or delay that can be represented as 100% ban-safe.
53. A count-only extension may perform less work than a full profile scraper, but it still generates automated authenticated search activity.
54. Use pause/resume, explicit error states, local-only storage, and conservative request scheduling as engineering safeguards—not as a guarantee against enforcement.
55. Do not ask users to paste li_at, JSESSIONID, CSRF tokens, passwords, or browser cookies into the extension.
56. Review data-protection obligations before combining follower-derived targeting with contact enrichment or outreach.

Scope boundary This document describes URL partitioning and validation. It does not provide a bypass for access controls, CAPTCHA, account restrictions, or LinkedIn's result ceiling.

14. Recommended implementation sequence

Phase	Build	Exit condition
Phase 1	URL codec + static local mock counts	Prove safe query rewriting
Phase 2	Visible-page count reader	Validate one URL at a time
Phase 3	Persistent queue + recursive planner	Split only over-cap nodes
Phase 4	Coverage accounting + unresolved states	Prevent false completeness claims
Phase 5	CSV exporter + compatibility test	Feed your existing 2,500-lead exporter
Phase 6	Small controlled live test	Verify follower filter and resume behavior

First milestone Start with a follower search that has one known over-cap industry bucket. The extension should accept the small industry children and recursively subdivide only the oversized one. That proves the architecture without generating thousands of unnecessary URLs.

Sources and evidence

[1] [Lobstr — Free LinkedIn Sales Navigator URL Splitter](#)

Public workflow, marketing claims, example counts, and facet selections.

[2] [Lobstr Help — Split Search in Sales Navigator](#)

Documented paid algorithm: 10/26/146 values, ~2,000 target, max 95, no recursion, ~50K guidance.

[3] [LinkedIn Help — Search results limit in Sales Navigator](#)

Official limit of 2,500 lead results across 100 pages.

[4] [LinkedIn Help — Automated activity on LinkedIn](#)

Policy statement on third-party software, scraping, browser extensions, and automation.

[5] [Microsoft Learn / LinkedIn — Industry Codes V2](#)

Current hierarchical industry taxonomy and IDs beyond the legacy list.

[6] User-provided Pasted text(11).txt

Local audit of 68 follower-search URLs; URL values and transient session context are not reproduced.

Evidence classification

Classification	Included evidence
Confirmed	Visible UI, generated cardinalities, generated URL shapes, supplied CSV structure
Vendor documented	Paid Split Search selection order, sizing estimate, non-recursion, limits
Strong inference	Free generator uses maintained static facet dictionaries and direct query rewriting
Not established	Lobstr's private source code, exact server/client division, complete follower coverage

Appendix A — Current static values observed

Industry: 68 values in the supplied/public output (listed in Section 3). Function: 26 generated values.

Seniority: 10 generated values with observed IDs:

320, 310, 300, 220, 210, 200, 130, 120, 110, 100

These values are snapshots of the current public generator, not a contract. A production extension should discover current facet options when possible and version any fallback dictionary.